claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-af636c5ecb7b6056d.jsonl`
- SHA-256: `58d1eb21772dff54ea117930fd03f9f3121daf38f267156ec43d3d5be295c3f5`
- Source modified: `2026-07-04T08:53:42+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_8c732492-138`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`

Transcript

1. user (2026-07-04T08:47:59.442Z)

```
You are fixing a code-review blocker on PR #469 (branch fix/highlights-signed-url) of
Khelsutra/badminton-highlight-indexer.
Your worktree: C:/Users/avidu/Projects/khelsutra-guru/wt-signed-download  (must be on
fix/highlights-signed-url at d3b0e64)

ENVIRONMENT (read carefully):
- Windows 11 box; use the PowerShell tool for shell commands (or Bash for POSIX one-liners).
Global Python 3.13.13 has the full offline dep set (pytest 9.0.3 works).
- Repo: Khelsutra/badminton-highlight-indexer. You work in an EXISTING git worktree (path given
below) that is already checked out on the PR branch at the PR head. Other sibling worktrees
exist for other PRs ? do NOT touch them, do NOT touch the main checkout at
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
- FIRST: run "git branch --show-current" and "git log --oneline -1" in your worktree and confirm
they match the branch+head stated below. If they do not match, STOP and report.
- Do NOT commit. Do NOT push. Do NOT run any gh/network write command. Leave your changes
UNCOMMITTED in the worktree; the orchestrator reviews, commits, and pushes.
- MANDATORY verification gates before you report done (run from the worktree root, report exact
results):
  1. FULL suite: python -m pytest tests/ -q -p no:cacheprovider   (~2-5 min; NEVER run just the
changed test file as your final gate ? the whole suite must be green; report exact "N passed, M
skipped" counts and any failures verbatim)
  2. Lint: python -m ruff check .   (CI pins ruff==0.15.16; if ruff is missing or a different
major, python -m pip install ruff==0.15.16)
  3. Types: python -m mypy backend training   (CI pins mypy==2.1.0; install if missing)
- Note: sibling agents run their own pytest suites concurrently on this box ? expect slower
wall-clock, and use -p no:cacheprovider exactly as given.
- CODE STYLE: match the repo ? rich docstrings that explain WHY and cite the design constraint
(see claim_due_job's existing docstring for tone); comments only where the code can't say it;
small functions. backend/main.py has an enforced line budget (tests/test_main_module_budget.py)
? avoid growing it.
- Your final message is consumed by the orchestrator via the structured schema. Report HONESTLY
? if a gate is red or you could not finish, say so in the schema fields; never claim green that
you didn't see.

REVIEWER BLOCKER (verbatim, from PR #469):
"One more issue: the new storage.signed_url() helper currently ignores the backend's configured
signed_url_ttl.
Even though the active storage config is now threaded into signed_reel_url(), the helper
signature defaults ttl=3600 and always passes that value into the backend:
    def signed_url(..., ttl: int = 3600, ...):
        return get_storage(...).signed_url(ref, ttl=ttl, method=method)
Both GCSStorage and S3Storage only fall back to their configured self._ttl when ttl is falsey
(ttl or self._ttl), so the configured TTL never gets a chance to apply for /api/highlights.
I reproduced locally on this head with an injected GCS client and config={'gcs':
```

{'signed_url_ttl': 123}}; calling storage.signed_url('gcs://bucket/highlights/vid/reel.mp4', config=..., method='GET') passed an expiration of 3600 seconds, not 123.
Suggested fix: make the helper's ttl optional/None and only pass an explicit ttl when the caller provides one, or pass ttl=0/None through so the backend uses its configured signed_url_ttl.
Please add a small regression test that /api/highlights / signed_reel_url() honors a non-default storage.gcs.signed_url_ttl (and ideally S3 too, since it has the same pattern)."

TASK: Make the module-level storage.signed_url() helper honor the backend's configured signed_url_ttl.
- File: backend/storage/__init__.py, signed_url() at ~line 205: change ttl: int = 3600 -> ttl: Optional[int] = None and pass it through unchanged; with ttl=None the backends' existing "ttl or self._ttl" fallback applies their configured signed_url_ttl (GCSStorage: backend/storage/gcs.py:76-78; S3Storage: backend/storage/s3.py:90-95). Update the helper docstring (None => the backend's configured signed_url_ttl; an explicit int still overrides).
- Check whether a StorageBackend base class / protocol declares signed_url with a ttl annotation (look in backend/storage/base.py or wherever the ABC lives) and keep annotations consistent so mypy stays green. Grep for ALL callers of signed_url (module helper and backend methods) to confirm nothing relied on the implicit 3600 default in a way that changes behavior you didn't intend; report what you found.
- orchestration.signed_reel_url() (backend/orchestration.py:703-713) already calls storage.signed_url(reel_uri, config=settings, method='GET') without ttl ? after the fix that path picks up the configured TTL automatically. Verify and say so.
- REGRESSION TESTS (required by reviewer; mimic the existing injected-client fixture patterns in tests/test_storage.py and the /api/highlights tests in tests/test_upload_download.py):
  1. Module helper + injected GCS client + config={'gcs': {'signed_url_ttl': 123}} -> the client sees expiration 123s (this is the reviewer's exact repro ? it must now pass).
  2. Same for S3 (config={'s3': {'signed_url_ttl': ...}} -> ExpiresIn honors it).
  3. signed_reel_url()//api/highlights honors a non-default storage.gcs.signed_url_ttl end-to-end.
  4. An explicit ttl passed to the helper still overrides the configured value (cheap, locks the contract).

When done, fill the structured output schema exactly and honestly.

2. user (2026-07-04T08:48:06.030Z)

fix/highlights-signed-url
d3b0e64 review: thread active storage config into mirror/exists/sign; extract helper (#463)

3. assistant (2026-07-04T08:48:16.637Z)

Worktree confirmed on `fix/highlights-signed-url` at `d3b0e64`. Now let me read the relevant files.

4. user (2026-07-04T08:48:16.765Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13
14      from __future__ import annotations
```

```
15
16      import os
17      import shutil
18      import tempfile
19      from typing import Any, Dict, Iterator, Optional
20
21      from backend.storage.base import StorageBackend
22      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24      __all__ = [
25          "StorageBackend",
26          "StorageRef",
27          "StorageStat",
28          "parse_uri",
29          "resolve_input_uri",
30          "get_storage",
31          "fetch",
32          "put",
33          "exists",
34          "signed_url",
35          "list_uris",
36          "acquire_local_input",
37          "cleanup_acquired_local_input",
38          "input_is_local",
39          "resolve_input_ref",
40      ]
41
42
43      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
44          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
45          if storage_config is None:
46              return {}
47          if hasattr(storage_config, "model_dump"):
48              return storage_config.model_dump()  # type: ignore[no-any-return]
49          return dict(storage_config)
50
51
52      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
53          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
54          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
55          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
56
57          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
58          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
59          sc = _storage_dict(storage_config)
60          uri = resolve_input_uri(
61              raw,
62              input_backend=sc.get("input_backend", "local"),
63              input_root=sc.get("input_root", ""),
64          )
65          return parse_uri(uri)
66
67
68      def input_is_local(raw: str, storage_config: Any = None) -> bool:
69          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
70          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
71          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
72          ``resolve_input_ref``)."""
```

```
        return resolve_input_ref(raw, storage_config).backend == "local"


    def acquire_local_input(
        raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
    ) -> str:
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The caller owns successful remote scratch copies; use
        ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return (
                ref.key
            )  # identity ? no temp, no copy, byte-identical to passing the path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        try:
            return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
        except Exception:
            shutil.rmtree(scratch, ignore_errors=True)
            raise


    def cleanup_acquired_local_input(
        raw: str, local_path: str, storage_config: Any = None
    ) -> None:
        """Delete a remote scratch dir produced by ``acquire_local_input``.

        Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
        into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
        finished hashing/validating/stitching/segmenting the returned file.
        """
        try:
            if input_is_local(raw, storage_config):
                return
        except ValueError:
            return
        if not local_path:
            return
        scratch = os.path.dirname(os.path.abspath(local_path))
        if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
            shutil.rmtree(scratch, ignore_errors=True)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
```

```
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage

            return GDriveStorage
        if name == "gdrive-api":
            from backend.storage.gdrive_api import GDriveApiStorage

            return GDriveApiStorage
        raise ValueError(
            f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
        )


    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
        )


    def put(
        local_path: "str | os.PathLike[str]",
        uri: str,
        *,
        config: Optional[Dict[str, Any]] = None,
        content_type: Optional[str] = None,
```

```
190     ) -> str:
191         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
192         ref = parse_uri(uri)
193         get_storage(ref.backend, config=config).put(
194             local_path, ref, content_type=content_type
195         )
196         return uri
197 
198 
199     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
200         """True if the object at ``uri`` exists (a bucket object or a local file)."""
201         ref = parse_uri(uri)
202         return bool(get_storage(ref.backend, config=config).exists(ref))
203 
204 
205     def signed_url(
206         uri: str,
207         *,
208         config: Optional[Dict[str, Any]] = None,
209         ttl: int = 3600,
210         method: str = "GET",
211     ) -> str:
212         """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
213         the app's response path (so a large object isn't streamed back through the server;
#463). Backends
214         without object signing (e.g. local) raise ``NotImplementedError``."""
215         ref = parse_uri(uri)
216         return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
217 
218 
219     def list_uris(
220         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
221     ) -> Iterator[str]:
222         """Yield the URI of every object under ``prefix_uri``."""
223         ref = parse_uri(prefix_uri)
224         for r in get_storage(ref.backend, config=config).list(ref):
225             yield r.uri
226 
```

5. user (2026-07-04T08:48:17.412Z)

```
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:697:    GCS
project/client/signed_url_ttl), or None. Threaded into storage calls so a non-default backend
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:713:
return storage.signed_url(reel_uri, config=settings, method="GET")
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\docs\PLATFORM_ARCHITECTURE.md:119:
def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\docs\PLATFORM_ARCHITECTURE.md:138:
(for compute) and `signed_url` (for the provider/browser), so the pipeline stops caring
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\base.py:56:    def
signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:25:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:28:
self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:76:    def
signed_url(self, ref, *, ttl=3600, method="GET") -> str:
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:82:        # so
a plain generate_signed_url raises "you need a private key". Fall back to the IAM signBlob
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:94:
return blob.generate_signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:103:
return blob.generate_signed_url(version="v4", expiration=exp, method=method)
```

```
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:34:
"signed_url",
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:205:def
signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:216:
return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl, method=method)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:30:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:33:
self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:90:    def
signed_url(self, ref, *, ttl=3600, method="GET") -> str:
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:92:
return self._client.generate_presigned_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:98:def
test_local_signed_url_raises():
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:100:
LocalStorage().signed_url(parse_uri("/x"))
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:197:    def
generate_presigned_url(self, op, Params, ExpiresIn):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:216:    url =
be.signed_url(ref, ttl=900)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:252:    def
generate_signed_url(self, version, expiration, method):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:307:    assert
be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:383:def
test_gdrive_signed_url_not_supported(tmp_path):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py:386:
GDriveStorage(mount_root=str(_mount(tmp_path))).signed_url(
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:305:def
test_highlights_redirects_to_signed_url_when_reel_in_bucket(env, monkeypatch):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:322:
monkeypatch.setattr(m.storage, "signed_url", _signed)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:326:
# the active storage settings are threaded into BOTH exists() and signed_url() (so a non-default
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py:327:
# backend / signed_url_ttl is honored), not a fresh default client.
```

6. user (2026-07-04T08:48:21.257Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6       no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8
9       from __future__ import annotations
10
11      import os
12      from abc import ABC, abstractmethod
13      from typing import Iterator
14
15      from backend.storage.ref import StorageRef, StorageStat
16
17
18      class StorageBackend(ABC):
19          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
20          worker dispatcher needs; the rest support listing/checkpointing/serving."""
21
```

```
        @abstractmethod
        def fetch_to_local(
            self,
            ref: StorageRef,
            dst: "str | os.PathLike[str] | None" = None,
            *,
            overwrite: bool = False,
        ) -> str:
            """Make the object available as a local file; return its local path. For the
            ``local`` backend this is an identity passthrough (the object is already
local)."""

        @abstractmethod
        def put(
            self,
            local_path: "str | os.PathLike[str]",
            ref: StorageRef,
            *,
            content_type: "str | None" = None,
        ) -> StorageRef:
            """Upload/copy a local file to ``ref``; return ``ref``."""

        @abstractmethod
        def exists(self, ref: StorageRef) -> bool: ...

        @abstractmethod
        def stat(self, ref: StorageRef) -> StorageStat: ...

        @abstractmethod
        def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
            """Yield a ``StorageRef`` per object under ``prefix``."""

        @abstractmethod
        def delete(self, ref: StorageRef) -> None: ...

        def signed_url(
            self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET"
        ) -> str:
            """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
            sign (``local`` cannot; Drive only emulates) ? callers must treat
            ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
            raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")

```

7. user (2026-07-04T08:48:22.288Z)

```
    """Google Cloud Storage backend ? first-class object buckets.

    Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
    install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
    (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
    never from config. A ``client`` may be injected for tests.
    """

    from __future__ import annotations

    from datetime import timedelta
    from importlib import import_module
    from typing import Any, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat
```

```

    class GCSStorage(StorageBackend):
        def __init__(
            self,
            *,
            client: Any = None,
            project: Optional[str] = None,
            signed_url_ttl: int = 3600,
            **_ignored,
        ) -> None:
            self._ttl = signed_url_ttl
            if client is not None:
                self._client = client
                return
            storage_module: Any = import_module("google.cloud.storage")
            self._client = storage_module.Client(project=project)

        def _blob(self, ref: StorageRef):
            return self._client.bucket(ref.bucket).blob(ref.key)

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os

            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
            return ref

        def exists(self, ref) -> bool:
            return bool(self._blob(ref).exists())

        def stat(self, ref) -> StorageStat:
            b = self._blob(ref)
            b.reload()
            return StorageStat(
                size=int(b.size or 0),
                modified=b.updated,
                etag=b.etag,
                content_type=b.content_type,
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
                yield StorageRef(
                    "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
                )

        def delete(self, ref) -> None:
            self._blob(ref).delete()

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            blob = self._blob(ref)
            exp = timedelta(seconds=ttl or self._ttl)
            creds = getattr(self._client, "_credentials", None)
            # Local-key signing (a service-account JSON key exposes a `signer`) works out of
the box. But
```

```
81              # on Cloud Run / GCE the ADC credentials come from the metadata server and have
NO private key,
82              # so a plain generate_signed_url raises "you need a private key". Fall back to
the IAM signBlob
83              # API by passing the SA email + a fresh access token ? which requires the
runtime SA to hold
84              # roles/iam.serviceAccountTokenCreator on ITSELF. Best-effort: if we can't build
that, defer to
85              # the default call (which will raise a clear error rather than silently mis-
sign).
86              if creds is not None and not hasattr(creds, "signer"):
87                  try:
88                      from google.auth.transport.requests import Request as _AuthRequest
89
90                      if not getattr(creds, "token", None):
91                          creds.refresh(_AuthRequest())
92                      sa_email = getattr(creds, "service_account_email", None)
93                      if sa_email and sa_email != "default":
94                          return blob.generate_signed_url(
95                              version="v4",
96                              expiration=exp,
97                              method=method,
98                              service_account_email=sa_email,
99                              access_token=creds.token,
100                         )
101                 except Exception:  # pragma: no cover - exercised only on real Cloud Run
creds
102                     pass
103             return blob.generate_signed_url(version="v4", expiration=exp, method=method)
104
```

8. user (2026-07-04T08:48:22.464Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12
13      from __future__ import annotations
14
15      import os
16      from typing import Any, Iterator, Optional
17
18      from backend.storage.base import StorageBackend
19      from backend.storage.ref import StorageRef, StorageStat
20
21
22      class S3Storage(StorageBackend):
23          def __init__(
24              self,
25              *,
26              client: Any = None,
27              endpoint_url: Optional[str] = None,
28              region: Optional[str] = None,
29              addressing_style: str = "auto",
```

```
            signed_url_ttl: int = 3600,
            **_ignored,
        ) -> None:
            self._ttl = signed_url_ttl
            if client is not None:
                self._client = client
                return
            import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
            from botocore.config import Config  # type: ignore

            self._client = boto3.client(
                "s3",
                endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
                region_name=region,
                config=Config(s3={"addressing_style": addressing_style}),
            )

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("S3Storage.fetch_to_local requires a dst path")
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._client.download_file(ref.bucket, ref.key, dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            extra = {"ContentType": content_type} if content_type else None
            self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
            return ref

        def exists(self, ref) -> bool:
            try:
                self._client.head_object(Bucket=ref.bucket, Key=ref.key)
                return True
            except Exception:  # noqa: BLE001 - any boto/client error means "not found /
unreachable"
                return False

        def stat(self, ref) -> StorageStat:
            h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
            return StorageStat(
                size=int(h.get("ContentLength", 0)),
                modified=h.get("LastModified"),
                etag=h.get("ETag"),
                content_type=h.get("ContentType"),
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            paginator = self._client.get_paginator("list_objects_v2")
            for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
                for obj in page.get("Contents", []) or []:
                    key = obj["Key"]
                    yield StorageRef(
                        "s3", prefix.bucket, key, f"s3://{prefix.bucket}/{key}"
                    )

        def delete(self, ref) -> None:
            self._client.delete_object(Bucket=ref.bucket, Key=ref.key)

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            op = "get_object" if method.upper() == "GET" else "put_object"
            return self._client.generate_presigned_url(
                op,
```

```
94                 Params={"Bucket": ref.bucket, "Key": ref.key},
95                 ExpiresIn=ttl or self._ttl,
96             )
97
```

9. assistant (2026-07-04T08:48:35.060Z)

Now let me look at the existing tests and orchestration code.

10. user (2026-07-04T08:48:36.478Z)

```
180             store = self.store
181
182             class _P:
183                 def paginate(self, Bucket, Prefix=""):
184                     yield {
185                         "Contents": [
186                             {"Key": k}
187                             for (b, k) in store
188                             if b == Bucket and k.startswith(Prefix)
189                         ]
190                     }
191
192             return _P()
193
194         def delete_object(self, Bucket, Key):
195             self.store.pop((Bucket, Key), None)
196
197         def generate_presigned_url(self, op, Params, ExpiresIn):
198             return f"https://s3.example/{Params['Bucket']}/{Params['Key']}?X-Amz-
Expires={ExpiresIn}"
199
200
201     def test_s3_backend_roundtrip_with_fake_client(tmp_path):
202         be = S3Storage(client=_FakeS3Client())
203         assert isinstance(be, StorageBackend)
204         src = tmp_path / "m.mp4"
205         src.write_bytes(b"VIDEO")
206         ref = parse_uri("s3://rally/videos/m.mp4")
207         be.put(str(src), ref, content_type="video/mp4")
208         assert be.exists(ref)
209         assert be.stat(ref).size == 5
210         dst = tmp_path / "dl" / "m.mp4"
211         assert be.fetch_to_local(ref, str(dst)) == str(dst)
212         assert dst.read_bytes() == b"VIDEO"
213         assert [r.uri for r in be.list(parse_uri("s3://rally/videos/"))] == [
214             "s3://rally/videos/m.mp4"
215         ]
216         url = be.signed_url(ref, ttl=900)
217         assert url.startswith("https://s3.example/rally/videos/m.mp4") and "900" in url
218         be.delete(ref)
219         assert not be.exists(ref)
220
221
222     def test_s3_exists_false_on_missing():
223         assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False
224
225
226     # ---------------------------------------------------------------- GCS (fake client)
227
228
229     class _FakeBlob:
230         def __init__(self, store, bucket, name):
231             self.store, self.bucket, self.name = store, bucket, name
232             self.size = self.updated = self.etag = self.content_type = None
```

```
        def upload_from_filename(self, src, content_type=None):
            with open(src, "rb") as f:
                self.store[(self.bucket, self.name)] = f.read()
            self.content_type = content_type

        def download_to_filename(self, dst):
            with open(dst, "wb") as f:
                f.write(self.store[(self.bucket, self.name)])

        def exists(self):
            return (self.bucket, self.name) in self.store

        def reload(self):
            self.size = len(self.store[(self.bucket, self.name)])

        def delete(self):
            self.store.pop((self.bucket, self.name), None)

        def generate_signed_url(self, version, expiration, method):
            return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"


    class _FakeGCSClient:
        def __init__(self):
            self.store = {}

        def bucket(self, name):
            return type(
                "_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name, key)}
            )()

        def list_blobs(self, bucket, prefix=""):
            return [
                _FakeBlob(self.store, bucket, k)
                for (b, k) in self.store
                if b == bucket and k.startswith(prefix)
            ]


    def test_gcs_backend_lazy_imports_real_client(monkeypatch):
        seen = {}

        class _Client:
            def __init__(self, *, project=None):
                seen["project"] = project

        def fake_import(name):
            seen["module"] = name
            return types.SimpleNamespace(Client=_Client)

        monkeypatch.setattr("backend.storage.gcs.import_module", fake_import)
        be = GCSStorage(project="rally-project")
        assert seen == {
            "module": "google.cloud.storage",
            "project": "rally-project",
        }
        assert isinstance(be._client, _Client)


    def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
        be = GCSStorage(client=_FakeGCSClient())
        src = tmp_path / "w.json"
        src.write_text("{}")
        ref = parse_uri("gcs://rally/weights/w.json")
```

```
298         be.put(str(src), ref)
299         assert be.exists(ref)
300         assert be.stat(ref).size == 2
301         dst = tmp_path / "dl" / "w.json"
302         be.fetch_to_local(ref, str(dst))
303         assert dst.read_text() == "{}"
304         assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] == [
305             "gcs://rally/weights/w.json"
306         ]
307         assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
308         be.delete(ref)
309         assert not be.exists(ref)
310
311
312     # ------------------------------------------------------------- Google Drive (local
mount)
313     # The backend reads a locally MOUNTED Google Drive (Google Drive for Desktop). A
tmp_path
314     # folder stands in for the mount's "My Drive" root ? no network, no Drive API, no extra
deps.
315
316
317     def _mount(tmp_path):
318         root = tmp_path / "My Drive"
319         (root / "Sharing").mkdir(parents=True)
```

11. user (2026-07-04T08:48:37.425Z)

```
680                 f"stitching.min_shot_count={stitching.min_shot_count}.",
681             )
682         return video, kept, out_name
683
684
685     def reel_object_uri(cfg, video_id: str, name: str) -> str:
686         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
687         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
688         mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
689         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
690         if not root or "://" not in root:
691             return ""
692         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
693
694
695     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
696         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
697         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
698         isn't silently constructed as a fresh default."""
699         sc = getattr(cfg, "storage", None)
700         return sc.model_dump() if sc is not None else None
701
702
703     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
704         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
705         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
706         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
707         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
```

```
708         reel_uri = reel_object_uri(cfg, video_id, name)
709         if not reel_uri:
710             return None
711         settings = _storage_settings(cfg)
712         if storage.exists(reel_uri, config=settings):
713             return storage.signed_url(reel_uri, config=settings, method="GET")
714         return None
715
716
717     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
718         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
719         and return the {status, filepath, download_url} result the SPA polls for. Returns a
720         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
721         so the SPA always gets a clean verdict instead of a worker-crash error."""
722         import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
723
724         notify = progress or (lambda stage: None)
```

12. user (2026-07-04T08:48:38.380Z)

```
280         client, db, tmp_path, cfg = env
281         db.add_video("vid", "/x.mp4", "processed", [])
282         blob = b"\x00\x00fake-mp4-bytes" * 100
283         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
284             fh.write(blob)
285         r = client.get("/api/highlights/vid/reel.mp4")
286         assert r.status_code == 200
287         assert r.content == blob
288         assert r.headers["content-type"].startswith("video/mp4")
289         assert "reel.mp4" in r.headers.get("content-disposition", "")
290
291
292     # --- #463: large-reel download via a signed URL (bypass Cloud Run's ~32 MiB response
cap) ---------
293
294
295     def test_reel_object_uri_cloud_vs_local():
296         from backend.orchestration import reel_object_uri
297
298         cloud = AppConfig.model_validate({"storage": {"output_root": "gs://b/out"}})
299         assert reel_object_uri(cloud, "vid", "reel.mp4") ==
"gs://b/out/highlights/vid/reel.mp4"
300         assert reel_object_uri(AppConfig(), "vid", "reel.mp4") == ""  # default output_root
"" ? local
301         local = AppConfig.model_validate({"storage": {"output_root": "/tmp/out"}})
302         assert reel_object_uri(local, "vid", "reel.mp4") == ""  # a local path is not a
bucket
303
304
305     def test_highlights_redirects_to_signed_url_when_reel_in_bucket(env, monkeypatch):
306         """A real reel exceeds Cloud Run's 32 MiB response cap; when it was mirrored to the
output bucket,
307         /api/highlights 307-redirects to a signed URL (client fetches bytes straight from
GCS) ? #463."""
308         client, db, _tmp, cfg = env
309         cfg.storage.output_root = "gs://b/out"
310         db.add_video("vid463", "gs://b/in/x.mp4", "processed", [])
311         seen = {}
312
313         def _exists(uri, **k):
314             seen["exists_cfg"] = k.get("config")
315             return uri == "gs://b/out/highlights/vid463/reel.mp4"
```

```
316
317         def _signed(uri, **k):
318             seen["sign_cfg"] = k.get("config")
319             return f"https://signed.example/{uri.rsplit('/', 1)[-1]}?sig=abc"
320
321         monkeypatch.setattr(m.storage, "exists", _exists)
322         monkeypatch.setattr(m.storage, "signed_url", _signed)
323         r = client.get("/api/highlights/vid463/reel.mp4", follow_redirects=False)
324         assert r.status_code == 307
325         assert r.headers["location"] == "https://signed.example/reel.mp4?sig=abc"
326         # the active storage settings are threaded into BOTH exists() and signed_url() (so a
non-default
327         # backend / signed_url_ttl is honored), not a fresh default client.
328         assert seen.get("exists_cfg") is not None and seen["exists_cfg"].get("output_root")
== "gs://b/out"
329         assert seen.get("sign_cfg") is not None and seen["sign_cfg"].get("output_root") ==
"gs://b/out"
330
331
332     def test_highlights_falls_back_to_local_when_no_bucket_reel(env, monkeypatch):
333         """No mirrored bucket object (or a signing hiccup) ? serve the local file, unchanged
behaviour."""
334         client, db, _tmp, cfg = env
335         cfg.storage.output_root = "gs://b/out"
336         db.add_video("vid463b", "gs://b/in/x.mp4", "processed", [])
337         monkeypatch.setattr(m.storage, "exists", lambda uri, **k: False)  # nothing in the
bucket
338         blob = b"local-reel" * 50
339         with open(os.path.join(cfg.output.output_dir, "reel.mp4"), "wb") as fh:
340             fh.write(blob)
341         r = client.get("/api/highlights/vid463b/reel.mp4")
342         assert r.status_code == 200
343         assert r.content == blob
344
```

13. assistant (2026-07-04T08:49:07.280Z)

Let me check how tests import things, the config model for storage, and the env fixture.

14. user (2026-07-04T08:49:08.951Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       The S3/GCS backends are exercised with injected FAKE clients (no boto3/google-cloud-
storage,
4       no network); the gdrive backend reads a locally MOUNTED Drive, exercised against a
tmp_path
5       that stands in for the mount ? all offline, matching the repo's ethos.
6       """
7
8       import os
9       import subprocess
10      import sys
11      import types
12
13      import pytest
14
15      from backend import storage
16      from backend.storage import (
17          StorageBackend,
18          fetch,
19          get_storage,
20          list_uris,
21          parse_uri,
22          put,
```

```
    )
    from backend.storage.gcs import GCSStorage
    from backend.storage.gdrive import GDriveStorage
    from backend.storage.local import LocalStorage
    from backend.storage.s3 import S3Storage

    # ------------------------------------------------------------- URI parsing


    @pytest.mark.parametrize(
        "uri,backend,bucket,key",
        [
            ("/data/x.mp4", "local", "", "/data/x.mp4"),
            ("rel/x.mp4", "local", "", "rel/x.mp4"),
            ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),  # Windows path: no '://'
            ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
            ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
            ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
            (
                "gs://rally-eu/w/model.json",
                "gcs",
                "rally-eu",
                "w/model.json",
            ),  # gs:// -> gcs
            ("gcs://rally/x", "gcs", "rally", "x"),
            (
                "gdrive://Sharing/clip.mp4",
                "gdrive",
                "",
                "Sharing/clip.mp4",
            ),  # path under My Drive (no bucket)
        ],
    )
    def test_parse_uri(uri, backend, bucket, key):
        ref = parse_uri(uri)
        assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)


    def test_parse_uri_rejects_unknown_scheme():
        with pytest.raises(ValueError):
            parse_uri("ftp://host/x")


    # ------------------------------------------------------------ local backend


    def test_local_fetch_is_identity_passthrough(tmp_path):
        src = tmp_path / "v.mp4"
        src.write_bytes(b"abc")
        # No dst => returns the source path unchanged (no copy): preserves today's
behaviour.
        assert fetch(str(src)) == str(src)


    def test_local_put_and_roundtrip(tmp_path):
        src = tmp_path / "in.csv"
        src.write_text("Frame,Visibility,X,Y\n")
        dst_uri = str(tmp_path / "out" / "copy.csv")
        put(str(src), dst_uri)
        assert (tmp_path / "out" / "copy.csv").read_text() == src.read_text()


    def test_local_exists_stat_list_delete(tmp_path):
        be = LocalStorage()
        (tmp_path / "a.txt").write_text("hello")
```

```
87          (tmp_path / "sub").mkdir()
88          (tmp_path / "sub" / "b.txt").write_text("hi")
89          assert be.exists(parse_uri(str(tmp_path / "a.txt")))
90          assert not be.exists(parse_uri(str(tmp_path / "nope.txt")))
91          assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
92          listed = sorted(r.key for r in be.list(parse_uri(str(tmp_path))))
93          assert listed == sorted([str(tmp_path / "a.txt"), str(tmp_path / "sub" / "b.txt")])
94          be.delete(parse_uri(str(tmp_path / "a.txt")))
95          assert not (tmp_path / "a.txt").exists()
96
97
98      def test_local_signed_url_raises():
99          with pytest.raises(NotImplementedError):
100             LocalStorage().signed_url(parse_uri("/x"))
101
102
103     def test_list_uris_helper(tmp_path):
104         (tmp_path / "x.txt").write_text("1")
105         (tmp_path / "y.txt").write_text("2")
106         uris = sorted(list_uris(str(tmp_path)))
107         assert all(u.startswith("local://") for u in uris)
108         assert len(uris) == 2
109
110
111     # ------------------------------------------------------------- registry / get_storage
112
113
114     def test_get_storage_defaults_to_local():
115         assert isinstance(get_storage(), LocalStorage)
116         assert isinstance(get_storage(config={"backend": "local"}), LocalStorage)
117
118
119     def test_get_storage_unknown_backend_raises():
120         with pytest.raises(ValueError):
121             get_storage("azure")
122
123
124     def test_gdrive_not_mounted_raises_with_remedy(monkeypatch):
125         # The scheme parses; if Drive isn't mounted (no auto-detected/configured root)
construction
126         # is a clear, actionable error ? install link + how to include the folder in syncing
+ the
127         # mount-root override ? rather than a cryptic failure downstream.
128         monkeypatch.delenv("GDRIVE_MOUNT_ROOT", raising=False)
129         assert parse_uri("gdrive://Sharing/x.mp4").backend == "gdrive"
130         with pytest.raises(RuntimeError) as ei:
```

15. user (2026-07-04T08:49:10.028Z)

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\khelsutra-guru\wt-signed-
download\backend\config.py. Note: your current working directory is
C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>
```

16. user (2026-07-04T08:49:16.429Z)

```
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\flywheel.py:164:
getattr(storage_cfg, "output_root", "") or "",
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:153:
(``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket (``storage.output_root``)
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:163:
out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:256:
out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:259:
"cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-bucket) "
```

```
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:688:
mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:689:    root
= (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\orchestration.py:697:    GCS
project/client/signed_url_ttl), or None. Threaded into storage calls so a non-default backend
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\presets.py:41:# these
bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:391:class
StorageConfig(BaseModel):
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:401:
output_root: str = ""
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:402:    # ---
Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output `backend`/`output_root`).
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:406:    # =>
``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with ``input_backend``
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:410:
input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\config\models.py:418:    #
Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:54:
config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported URI scheme ?
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:62:
input_backend=sc.get("input_backend", "local"),
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py:83:
non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the local
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:25:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py:28:
self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\ref.py:76:    raw:
str, *, input_backend: str = "local", input_root: str = ""
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\ref.py:85:      4. A
bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` + ``"bucket/x"`` ?
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\ref.py:89:    So the
default (``input_backend="local"``, ``input_root=""``) is an identity passthrough for every
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\ref.py:103:
input_backend and input_backend != "local"
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\ref.py:105:
return f"{input_backend}://{s}"
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:30:
signed_url_ttl: int = 3600,
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py:33:
self._ttl = signed_url_ttl
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\capabilities.py:69:
if sc.get("backend") == name or sc.get("input_backend") == name:
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\capabilities.py:76:
sc.get("output_root") or "",
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\utils\workspace.py:48:
workspace_root: str, output_root: str, output_dir: str
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\utils\workspace.py:51:
``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-breaking)."""
C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\utils\workspace.py:54:
or (output_root or "").strip()
```

17. user (2026-07-04T08:49:24.826Z)

```
380         # application's audience tag. NO secrets here ? these are public identifiers.
381         edge_auth_enabled: bool = False
382         cf_team_domain: str = (
383             ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
384         )
385         cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
386         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
```

```
import in
387         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
388         # OFF ? Cf-Access is a header, not a cookie.)
389
390
391     class StorageConfig(BaseModel):
392         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
393         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
394         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
395         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
396         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
397         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
398
399         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
400         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
401         output_root: str = ""
402         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
403         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
404         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
405         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
406         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
407         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
408         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
409         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
410         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411         input_root: str = ""
412         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
413         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
414         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
415         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
416         # the legacy flat layout is used unchanged.
417         single_folder_per_video: bool = False
418         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
419         workspace_root: str = ""
420         # Cloud namespace prefix under the root so per-video folders sit tidily beside
421         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
422         workspace_prefix: str = "workspaces"
423         local: dict[str, Any] = Field(default_factory=dict)
424         s3: dict[str, Any] = Field(default_factory=dict)
425         gcs: dict[str, Any] = Field(default_factory=dict)
426         gdrive: dict[str, Any] = Field(default_factory=dict)
427         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
428         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
429         gdrive_api: dict[str, Any] = Field(default_factory=dict)
430
431
```

```
432     class DeploymentConfig(BaseModel):
433         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
434         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
435
436         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
437         behaves exactly as before this section existed. A profile is opt-in (``config.json``
438         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
439         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
```

18. assistant (2026-07-04T08:49:44.051Z)

Let me check the `env` fixture and imports in test_upload_download.py.

19. user (2026-07-04T08:49:45.780Z)

```
1       """Chunked upload (B2) + highlight download (B3) ? PR-2 of the UI alpha plan.
2
3       Contract under test:
4       - POST /api/upload/init ? {upload_id}; extension allowlist + size caps enforced
5       - PUT /api/upload/{id}/part/{n} ? strictly sequential parts; per-part/total caps abort
(413)
6       - POST /api/upload/{id}/complete ? assembled file, MD5 == compute_video_id, no overwrite
7       - DELETE /api/upload/{id} ? abort removes partial state
8       - GET /api/highlights/{video_id}/{filename} ? streams from output_dir; traversal-proof
9       """
10
11      import os
12      from unittest.mock import patch
13
14      import pytest
15
16      pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly where absent
17      from fastapi.testclient import TestClient
18
19      import backend.main as m
20      from backend.api import uploads as uploads_api
21      from backend.config.models import AppConfig
22      from backend.infrastructure.database import Database
23      from backend.utils.hashing import compute_video_id
24
25
26      @pytest.fixture
27      def env(tmp_path, monkeypatch):
28          db = Database(str(tmp_path / "t.db"))
29          cfg = AppConfig()
30          cfg.output.output_dir = str(tmp_path / "out")
31          cfg.security.upload_dir = str(tmp_path / "up")
32          os.makedirs(cfg.output.output_dir, exist_ok=True)
33          monkeypatch.setattr(
34              uploads_api, "_uploads", {}
35          )  # isolate upload state per test (registry moved to backend.api.uploads)
36          app = m.create_app(cfg, db)
37          client = TestClient(app)
38          return client, db, tmp_path, cfg
39
40
41      def _init(client, filename="match.mp4", total=None):
42          body = {"filename": filename}
43          if total is not None:
44              body["total_size_bytes"] = total
45          return client.post("/api/upload/init", json=body)
```

```
46
47
48      # --- init ------------------------------------------------------------------------
49
50
51      def test_init_rejects_bad_extension(env):
52          client = env[0]
53          r = _init(client, "malware.exe")
54          assert r.status_code == 400
55          assert "not allowed" in r.json()["detail"]
56
57
58      @pytest.mark.parametrize(
59          "filename,stored",
60          [
61              ("match.MP4", "match.MP4"),
62              ("match?MP4", "match.MP4"),
63              ("match.MP4.", "match.MP4"),
64              ("match.MP4 ", "match.MP4"),
65          ],
66      )
67      def test_init_accepts_normalized_mp4_names(env, filename, stored):
68          client, db, tmp_path, cfg = env
69          r = _init(client, filename)
70          assert r.status_code == 200
71          upload_id = r.json()["upload_id"]
72          assert uploads_api._uploads[upload_id]["filename"] == stored
73
74
75      def test_init_rejects_traversal_filename(env):
76          client = env[0]
77          assert _init(client, "../evil.mp4").status_code == 400
78          assert _init(client, "a/b.mp4").status_code == 400
79
80
81      def test_init_rejects_oversize_declared_total(env):
82          client, db, tmp_path, cfg = env
83          cfg.security.max_upload_mb = 1
84          r = _init(client, total=2 * 1024 * 1024)
85          assert r.status_code == 413
86
87
88      def test_init_507_when_disk_too_full_for_declared_total(env):
89          # P2 (D11): a declared upload that won't fit fails fast with 507 (not a mid-upload
crash).
90          import types
91
92          from backend.utils import freespace
93
94          client = env[0]
95          with patch.object(
96              freespace.shutil,
97              "disk_usage",
98              return_value=types.SimpleNamespace(total=0, used=0, free=1 * 1024 * 1024),
99          ):
100             r = _init(client, total=50 * 1024 * 1024)
101         assert r.status_code == 507
102         assert "space" in r.json()["detail"].lower()
103
104
105     def test_init_ok_when_disk_has_room(env):
106         import types
107
108         from backend.utils import freespace
109
```

```
110         client = env[0]
```

20. user (2026-07-04T08:50:02.133Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\__init__.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

21. user (2026-07-04T08:50:05.913Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\base.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

22. user (2026-07-04T08:50:07.312Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\gcs.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

23. user (2026-07-04T08:50:08.692Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\backend\storage\s3.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

24. assistant (2026-07-04T08:50:13.711Z)

```
Now the regression tests. First, make the fake GCS signer expose the expiration so tests can
assert on it.
```

25. user (2026-07-04T08:50:16.761Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

26. user (2026-07-04T08:50:27.477Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_storage.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

27. user (2026-07-04T08:50:44.942Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\wt-signed-download\tests\test_upload_download.py
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

28. user (2026-07-04T08:50:55.701Z)

```
.........................................................              [100%]
59 passed in 3.87s
```

29. assistant (2026-07-04T08:51:01.381Z)

```
Targeted tests pass. Now the full gates.
```

30. user (2026-07-04T08:51:03.396Z)

```
Command running in background with ID: b3ll3lmes. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\b3ll3lmes.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

31. user (2026-07-04T08:51:10.623Z)

```
ruff 0.15.16
All checks passed!
```

32. user (2026-07-04T08:51:23.584Z)

```
mypy 2.1.0 (compiled: yes)
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 147 source files
```

33. assistant (2026-07-04T08:51:31.470Z)

Lint and types are green. While the suite runs, let me review the final diff.

34. user (2026-07-04T08:51:33.382Z)

```
 M backend/storage/__init__.py
 M backend/storage/base.py
 M backend/storage/gcs.py
 M backend/storage/s3.py
 M tests/test_storage.py
 M tests/test_upload_download.py
 backend/storage/__init__.py   |  8 ++++++--
 backend/storage/base.py       |  6 ++++--
 backend/storage/gcs.py        |  2 +-
 backend/storage/s3.py         |  2 +-
 tests/test_storage.py         | 38 +++++++++++++++++++++++++++++++++++++-
 tests/test_upload_download.py | 29 +++++++++++++++++++++++++++++
 6 files changed, 78 insertions(+), 7 deletions(-)
```

35. user (2026-07-04T08:51:43.693Z)

```
1       ......................ss.............................................. [  4%]
2       .........
```

36. user (2026-07-04T08:52:07.779Z)

Command running in background with ID: bipeauzfp. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\bipeauzfp.output. You will be notified when it completes. To check interim output, use Read on that file path.

37. assistant (2026-07-04T08:52:14.968Z)

Waiting for the full suite to complete; I'll report the gates once the background run finishes.

38. user (2026-07-04T08:52:15.107Z)

[structured-output-enforce] You MUST call the StructuredOutput tool to complete this request.
Call this tool now.

39. user (2026-07-04T08:52:22.152Z)

```
1       ......................ss.............................................. [  4%]
2       ............................................................s........... [  8%]
3       ..................................................................... [ 13%]
4       ..................................................................... [ 17%]
5       ..........
```

40. assistant (2026-07-04T08:52:32.518Z)

The suite is still running; I must wait for it before reporting. Let me block on completion in the foreground.

41. user (2026-07-04T08:53:12.324Z)

```
tests/test_api_endpoints.py::test_process_remote_input_reports_use_durable_output_dir
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gs://bucket/video
s/x.mp4]
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gcs://bucket/vide
os/x.mp4]
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1633 passed, 10 skipped, 8 warnings in 126.22s (0:02:06)
```

42. user (2026-07-04T08:53:42.554Z)

```
Structured output provided successfully
```